

Building Serverless Data Pipeline with AWS Lambda Amazon Kinesis Data Streams and Amazon Data Firehose

Contents

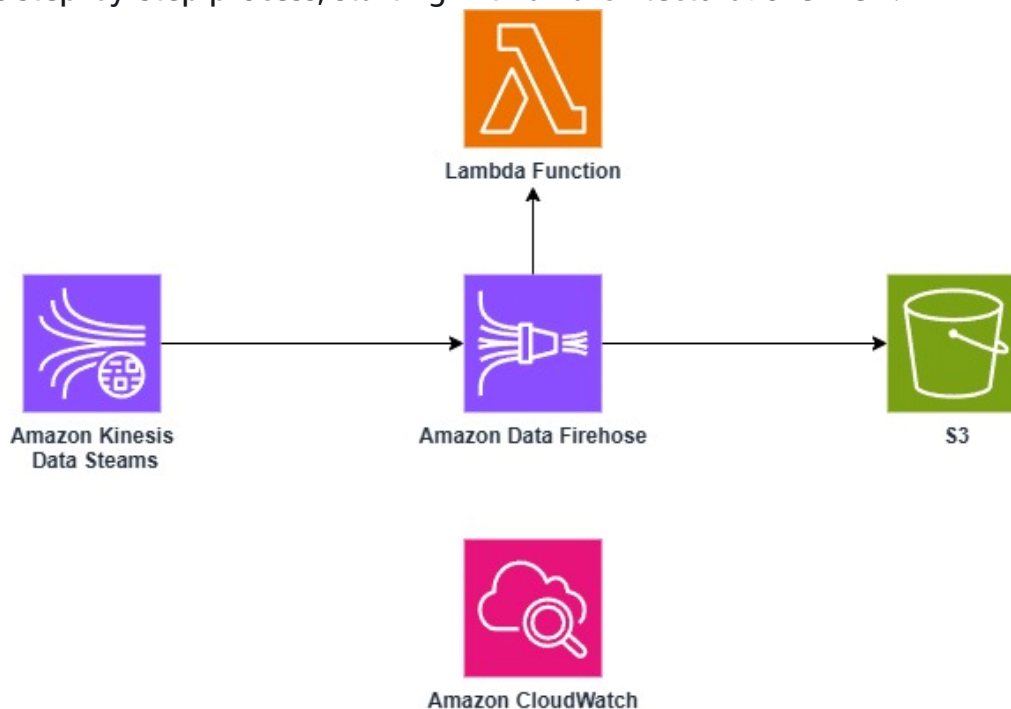
Building Serverless Data Pipeline with AWS Lambda Amazon Kinesis Data Streams and Amazon Data Firehose	1
Introduction	1
Serverless Data Pipelines with AWS Lambda, Amazon Kinesis Data Streams and Amazon Data Firehose	2
Real-world example	3
Hands-On Exercise: Building a Serverless Data Ingestion and Processing Pipeline....	4
Step 1: Provisioning an Amazon Kinesis Data Stream	4
Step 2: Developing the Lambda Function	4
Step 3: Test Lambda Function	6
Step 4: Provision Storage for Processed Data	7
Step 5: Provision an Amazon Data Firehose	7
Step 6: Test and Monitor the Pipeline	10
Pedagogic Approach	11

Introduction

Serverless architectures and real-time data processing are rapidly becoming essential skills in the tech industry. This article guides through the process of building a scalable, serverless data pipeline using Amazon Kinesis Data Streams, Amazon Data Firehose, and Lambda within the AWS Academy Learner Lab. The goal is to provide a practical understanding of how to ingest near real-time data, process it using serverless compute, and store the transformed data in a storage solution, all without the need to manage any underlying infrastructure.

Serverless Data Pipelines with AWS Lambda, Amazon Kinesis Data Streams and Amazon Data Firehose

The AWS Academy Learner Lab provides access to AWS Lambda, Amazon Kinesis Data Streams, Amazon Data Firehose, and S3 to set up a serverless data pipeline. Let's dive into the step-by-step process, starting with an architectural overview:



1. **Amazon Kinesis Data Stream:** This is the entry point of the data pipeline, where real-time data is ingested and stored. Kinesis Data Streams can continuously capture gigabytes of data per second from hundreds of thousands of sources. The collected data is available in milliseconds to allow real-time analytics use cases, such as real-time dashboards, real-time anomaly detection, and dynamic pricing.
2. **Amazon Data Firehose:** This is used to acquire, transform, and deliver data to the data storage. Amazon Data Firehose continuously process the stream, automatically scale based upon data, and delivers it within seconds. Firehose buffers incoming data and invokes the specified Lambda function with each buffered batch asynchronously. The transformed data is sent back from Lambda to Firehose for buffering.
3. **AWS Lambda Function:** Amazon Data Firehose invokes a Lambda function to transform incoming source data and deliver the transformed data to destinations. You enable Amazon Data Firehose data transformation when you create your Firehose stream.

4. **Data Storage:** The processed data is then stored in either Amazon S3 (for data lake storage) or Amazon OpenSearch (for low-latency access).

Real-world example

Let's consider a real-world scenario of an online store that receives thousands of orders every hour. The store wants to analyze sales data in near real-time to make quick decisions about inventory, pricing, and marketing. Here's how a serverless data pipeline helps achieve this goal:

1. **Amazon Kinesis Data Streams:** As customers place orders on the online store, Kinesis Data Streams receives and stores this data. It's designed to handle the large volume of incoming orders in real-time, ensuring that no sales data is lost even during peak shopping times.
2. **Amazon Data Firehose:** Firehose is configured to deliver data it receives from Kinesis Data Streams into Amazon S3 by performing data transformation using a Lambda function. After Lambda processes the order data, Firehose takes over. It reliably delivers the processed data to long-term storage. For the online store, Firehose ensures that all the analyzed sales data is safely stored for future reference and deeper analysis.
3. **AWS Lambda:** Amazon Data Firehose invokes a Lambda function to calculate total sales, identify popular items, or flag low-stock products. Lambda scales automatically with the volume of orders, so it can handle both quiet periods and sudden spikes in sales without any manual intervention.
4. **Amazon S3 (Simple Storage Service):** S3 is where the online store keeps all its processed sales data long-term. This could include daily sales reports, inventory updates, and customer behavior analytics. S3 allows the store to keep years of historical data, which can be important for identifying long-term trends.

This pipeline provides the online store with real-time insights into their sales. For example, if a particular product starts selling rapidly, the store can immediately order more stock or adjust pricing. The system does this automatically, without requiring staff to manually run reports or manage complex IT infrastructure.

By implementing this serverless data pipeline, the online store creates a tool that can handle large amounts of real-time sales data, process it automatically, and store it for future analysis. This approach is cost-effective and requires minimal maintenance, allowing the store to focus on using the insights gained to improve their business operations.

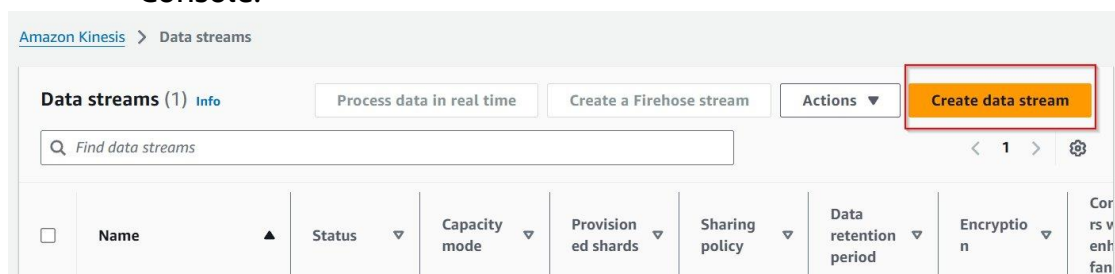
Let's dive into the step-by-step process of building this serverless data pipeline with the AWS Academy Learner Lab:

Hands-On Exercise: Building a Serverless Data Ingestion and Processing Pipeline

To gain practical experience, follow these steps to build a serverless data pipeline using the AWS Console with the Learner Lab:

Step 1: Provisioning an Amazon Kinesis Data Stream

- Navigate to the **Amazon Kinesis** service in the AWS Management Console.



- Create a new Kinesis Data Stream with a name of your choice (e.g., "my-data-stream"), set capacity mode to **Provisioned** and enter **1** for number of **shards**.
- Review the settings and click "Create data stream" to provision the Kinesis Data Stream.

Step 2: Developing the Lambda Function

- On AWS Console, search for the **Lambda** and open the Lambda Console.
- Create a new function with name of your choice ("my-data-transformer"), select a **Runtime** environment (**Python 3.13**). Change the default execution role and select **Lab Role**. Click **Create function** to create a new function.

Basic information

Function name
Enter a name that describes the purpose of your function.
my-data-transformer
Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime Info
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.
Python 3.13

Architecture Info
Choose the instruction set architecture you want for your function code.
☒ x86_64
☐ arm64

Permissions Info
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

▼ **Change default execution role**

Execution role
Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#).
☐ Create a new role with basic Lambda permissions
☒ Use an existing role
☐ Create a new role from AWS policy templates

Existing role
Choose an existing role that you've created to be used with this Lambda function. The role must have permission to upload logs to Amazon CloudWatch Logs.
LabRole
[View the LabRole role](#) on the IAM console.

- c. On Lambda Console, select the **Configuration** tab, click on "General Configuration" and click on **Edit**. Change the **Timeout** to **1 min**.

Code | Test | Monitor | **Configuration** | Aliases | Versions

General configuration Info Edit

Description -	Memory 128 MB	Ephemeral storage 512 MB
Timeout 0 min 3 sec	SnapStart Info None	

Triggers
Permissions
Destinations
Function URL
Environment variables

- d. On Lambda Console, select the **Code** tab, and replace the existing code in **lambda_function.py** file with:

```
import base64
import boto3

def lambda_handler(event, context):
    output = []
    #iterate over records received, base64 decode, custom
    process and add to output
    for record in event['records']:
        try:
            # Kinesis data is base64 encoded so decode here
            payload =
            base64.b64decode(record['data']).decode('utf-8')
```

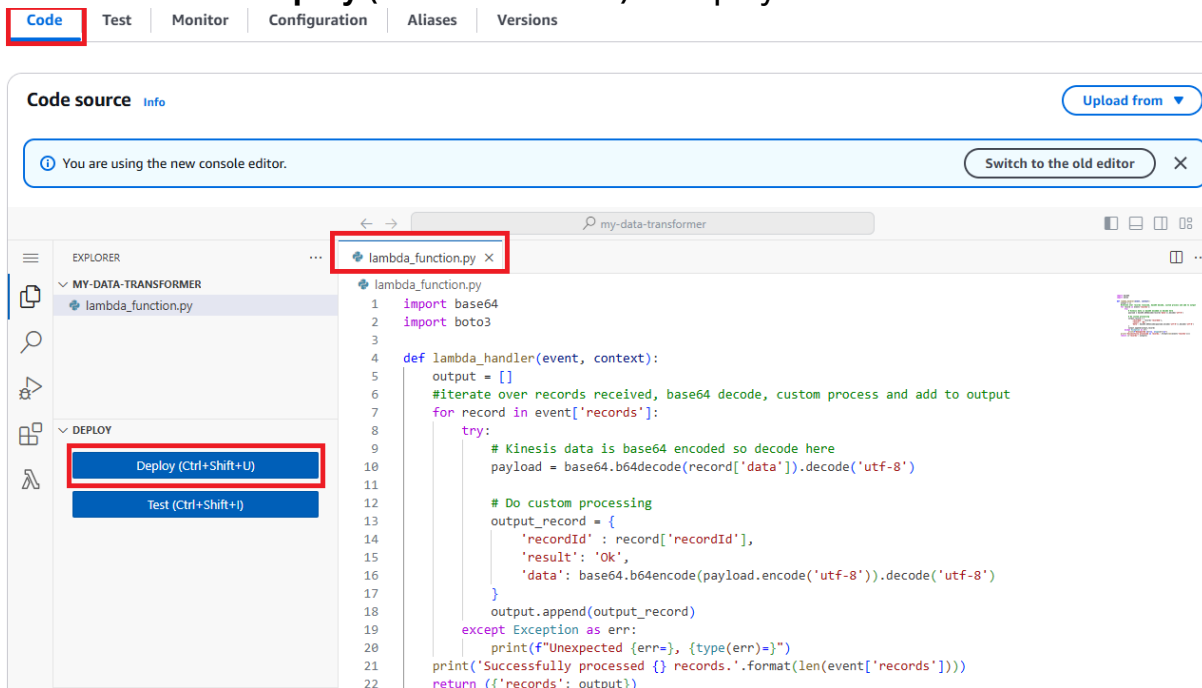
```

# Do custom processing
output_record = {
    'recordId' : record['recordId'],
    'result': 'Ok',
    'data': base64.b64encode(payload.encode('utf-
8')).decode('utf-8')
}
output.append(output_record)
except Exception as err:
    print(f"Unexpected {err=}, {type(err)=}")
print('Successfully processed {}
records.'.format(len(event['records'])))
return ({'records': output})

```

This Lambda function transforms records from Amazon Data Firehose. It receives a batch of order records, decodes them from base64 encoding, and prepares them for custom processing. While the current implementation is a placeholder, you would replace it with a specific logic to analyze sales data, such as calculating totals, checking inventory levels, or identifying popular products.

e. Click **Deploy** (or CTRL+Shift+U) to deploy the Lambda Function.



Step 3: Test Lambda Function

- Click on **Test** tab, enter event name "firehose-test-event", select the **Template "Kinesis Data Firehose"** ("kinesis-kinesis-firehose") and click **Save**.
- Click **Test** to run the test and validate the response.

- b. Click **Create Firehose stream** and choose the source "Amazon Kinesis Data Streams" and Destination "Amazon S3". Use the default stream name.

Create Firehose stream [Info](#)

► Amazon Data Firehose: How it works

Choose source and destination
Specify the source and the destination for your Firehose stream. You cannot change the source and destination of your Firehose stream once it has been created.

Source [Info](#)
Amazon Kinesis Data Streams ▼

Destination [Info](#)
Amazon S3 ▼

- c. Browse the **Kinesis data stream** and choose your data stream (in this example "my-data-stream")
- d. Check **Turn on data transformation**, browse AWS Lambda Function and choose your Lambda function (in this example "my-data-transformer").

Source settings

Kinesis data stream

arn:aws:kinesis:us-east-1:stream/my-data-stream

Browse

Create

Format: arn:aws:kinesis:[Region]:[AccountId]:stream/[StreamName]

Transform and convert records - optional

Configure Amazon Data Firehose to transform and convert your record data.

Transform source records with AWS Lambda

Amazon Data Firehose can invoke an AWS Lambda function to transform, filter, decompress, convert and process your source data records. The specified AWS Lambda function can also be used to provide data partitioning keys for the incoming source data before its delivery to the specified destination.

☒ Turn on data transformation

AWS Lambda function

Version or alias

arn:aws:lambda:us-east-1:

\$LATEST

Browse

Create function

Format: arn:aws:lambda:[Region]:[AccountId]:function:[FunctionName]

Buffer size

The AWS Lambda function has a 6 MB invocation payload quota. Your data can expand in size after it's processed by the AWS Lambda function. A smaller buffer size allows for more room should the data expand after processing.

1

MB

Minimum: 0.2 MB, maximum: 3 MB.

Buffer interval

The period of time during which Amazon Data Firehose buffers incoming data before invoking the AWS Lambda function. The AWS Lambda function is invoked once the value of the buffer size or the buffer interval is reached.

60

seconds

Minimum: 0 seconds, maximum: 900 seconds.

Convert record format

Data in Apache Parquet or Apache ORC format is typically more efficient to query than JSON. Amazon Data Firehose can convert your JSON-formatted source records using a schema from a table defined in [AWS Glue](#). For records that aren't in JSON format, create a Lambda function that converts them to JSON in the Transform source records with AWS Lambda section above.

☐ Enable record format conversion

Decompress source records from Amazon CloudWatch Logs - new

- e. Browse S3 bucket in **Destination settings** and choose the destination bucket (in this example "my-academy-lab-data-storage").
- f. Click on **Advanced settings** and navigate to **Service access**. Select **Choose existing IAM role** and **LabRole** from the existing IAM roles.

Service access

Amazon Data Firehose uses this IAM role for all the permissions that the Firehose stream needs. To specify different roles for the different permissions, use the API or the CLI.

- ☐ Create or update IAM role **KinesisFirehoseServiceRole-KDS-S3-UWv57-us-east-1-1733171995166**
Creates a new role or updates an existing one and adds the required policy to it, and enables Amazon Data Firehose to assume it.

☒ **Choose existing IAM role**

The role that you choose must have policies that include the permissions that Amazon Data Firehose needs.

If you choose an existing IAM role for Amazon Data Firehose, make sure the chosen IAM role has a proper trust policy and permissions required for your source and destination. For more information, see [Controlling Access with Amazon Data Firehose](#).

Existing IAM roles

Only IAM roles with the required trust policy are available for selection.

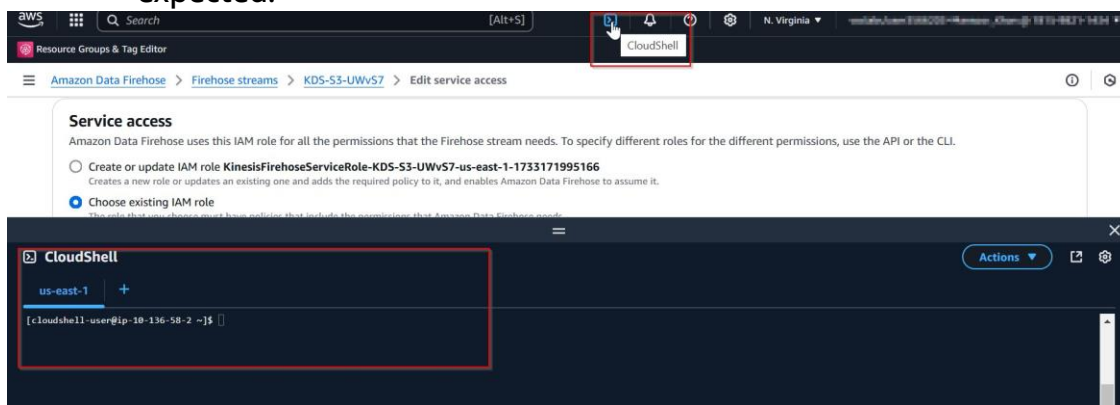
LabRole

[View LabRole](#) in the IAM console.

g. Click **Create Firehose Stream**.

Step 6: Test and Monitor the Pipeline

- a. Use AWS CLI through CloudShell to put sample records into the Kinesis Data Stream and verify that the data is being processed and stored as expected.



```
aws kinesis put-record --stream-name my-data-stream \
--data "Test Record" \
--partition-key uuidgen \
--cli-binary-format raw-in-base64-out
```

Important: Make sure to replace **my-data-stream** by your stream name.

Note: Amazon Data Firehose persists data to Amazon S3 near real-time. The delay is governed by the buffer size (default 1MB) or buffer duration (default 60 seconds). Wait for at least this duration to verify the records on Amazon S3.

- b. Verify records are stored in your S3 bucket (in this example "my-academy-lab-data-storage").

Amazon S3 > Buckets > my-academy-lab-data-storage

my-academy-lab-data-storage Info

Objects Properties Permissions Metrics Management Access Points

Objects (5) Info Copy S3 URI Copy URL Download Open Delete Actions Create folder Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	shardid-000000000000:49545115243490985018280067714973144582180062593244200961	-	November 2, 2024, 16:14:15 (UTC-05:00)	26.0 B	Standard

- c. Verify records are processed by Amazon Data Firehose. The CloudWatch Dashboard displays **Records read from Kinesis Data Stream** and **Records Delivered to S3** for all the records persisted to the S3 bucket.

KDS-S3-UWvS7 Info Delete Firehose stream

Firehose stream details

Status: Active

Destination: Amazon S3

Source: Amazon Kinesis Data Streams

ARN: arn:aws:firehose:us-east-1:1434:deliverystream/KDS-S3-UWvS7

Data transformation: Enabled

Dynamic partitioning: Not enabled

Creation time: November 19, 2024 at 11:37 CST

Error logs status: 1 Destination error logs

Test with demo data Ingest simulated data to test the configuration of your Firehose stream. Standard Amazon Data Firehose charges apply.

Monitoring Configuration Destination error logs

Firehose stream metrics Info

1h 3h 12h 1d 3d 1w Custom (8h) Local timezone Add to dashboard

Records read from Kinesis Data Streams (Sum)

Count

3

2

1

Bytes read from Kinesis Data Streams (Sum)

Bytes

30

20

10

DescribeStream operations throttled (Average)

Count

1

0.8

0.6

0.4

0.2

- d. Monitor the pipeline's execution by checking the logs in Amazon CloudWatch using the AWS Console. Click on **Log groups**, select the Log group for the **Kinesis Firehose** and click on **Delivery Events** to view the logs.

Pedagogic Approach

Provide clear, step-by-step instructions and sample code to guide your students through the hands-on exercise. Encourage them to explore the various data processing and storage options, and challenge them to come up with their own use cases and enhancements to the pipeline.

Through this hands-on activity, students will gain practical experience in building a serverless data pipeline using AWS Services, simulating a real-world scenario of

processing online store orders. They will learn how to integrate various AWS components, including Kinesis Data Streams, Lambda, and S3, to create a scalable and efficient data processing system. This exercise will provide students with valuable insights into cloud computing, real-time data processing, and the principles of serverless architecture, preparing them for the challenges of modern data engineering and cloud-based solutions in the business world.